BLOOD BANK MANAGEMENT SYSTEM: A ROLE-BASED FULL-STACK PLATFORM FOR REAL-TIME INVENTORY, ABO/Rh COMPATIBILITY MATCHING, AND EMERGENCY DONOR ALERTING

PHASE I REPORT

Submitted by

BHARATH KUMAR M	**230701053**
AKSHAY KUMAR S	**230701021**
BERNIEO FATIM A	**230701051**
ASHISH P SHAJI	**230701041**

in partial fulfillment for the award of the degree
of

BACHELOR OF ENGINEERING
IN
COMPUTER SCIENCE AND ENGINEERING





RAJALAKSHMI ENGINEERING COLLEGE, CHENNAI
ANNA UNIVERSITY: CHENNAI 600 025

OCTOBER 2026

ANNA UNIVERSITY, CHENNAI

BONAFIDE CERTIFICATE

Certified that this Report titled **"BLOOD BANK MANAGEMENT SYSTEM: A ROLE-BASED FULL-STACK PLATFORM FOR REAL-TIME INVENTORY, ABO/Rh COMPATIBILITY MATCHING, AND EMERGENCY DONOR ALERTING"** is the bonafide work of **BHARATH KUMAR M (230701053), AKSHAY KUMAR S (230701021), BERNIEO FATIM A (230701051),** and **ASHISH P SHAJI (230701041)** who carried out the work under my supervision. Certified further that to the best of my knowledge the work reported herein does not form part of any other thesis or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidates.

SIGNATURE
Dr. Manoranjini J, Ph.D.
Professor and Head,
Department of Computer Science and Engineering,
Rajalakshmi Engineering College,
Chennai – 602 105.

SIGNATURE
Mrs. Anitha M, M.E.,
Assistant Professor,
Department of Computer Science and Engineering,
Rajalakshmi Engineering College,
Chennai – 602 105.

Submitted to Project Viva-Voce Examination held on _____

Internal Examiner **External Examiner**

ABSTRACT

Blood is a perishable, life-critical commodity, and late visibility into stock or a mismatched transfusion can endanger patients. Most blood banks today operate under paper registers, spreadsheet inventory logs, single-branch bespoke software, or ad-hoc phone and chat tools, lacking cross-facility visibility, systematic ABO/Rh verification prior to issuance, or audit trail capabilities. This report describes the Blood Bank Management System (BBMS): a role-based, enterprise web platform implemented in Spring Boot 3 (Java 17), React 18, and MySQL 8 that digitizes the donation lifecycle from eligibility screening and blood-drive intake through hospital verification, ABO/Rh-specific request fulfillment, transfusion issuance, unit-level shelf-life tracking, and audit logging.

In addition to the architecture and data model of the twelve-module system, this report evaluates three internal mechanisms experimentally: first, the donor eligibility engine, whose exact decision procedure has been tested against a 622-case boundary-condition suite covering every rule threshold (age, weight, hemoglobin, cooldown, and health indicators) and demonstrated 100% agreement with independently derived expected classifications; second, the ABO/Rh compatibility engine, which has been exhaustively validated against all 64 recipient-donor group pairings in the AABB reference matrix (100% agreement) and demonstrated correct ranking behavior under 200 randomized inventory configurations; and third, the inventory layer's optimistic locking concurrency control (Hibernate @Version), which has been tested under true concurrency through OS-level parallel process invocation: at 200 simultaneous transactions against shared stock, a naive read-modify-write algorithm lost a mean of 132 of 200 concurrent updates (66%), while the optimistic locking pattern lost a mean of 0.2 updates at the cost of a mean 1,558 compare-and-swap retries.

Furthermore, BBMS integrates a dual-channel emergency alerting gateway powered by Twilio Voice and SMS REST APIs, paired with a real-time WebSocket/STOMP broadcast pipeline to propagate critical shortages immediately across subscribed hospital and blood bank dashboards. By enforcing a strict zero-direct-mutation invariant on blood stock and logging every state alteration to an immutable audit trail, the platform provides a clinically secure, auditable, and resilient foundation for modern healthcare information infrastructure

ACKNOWLEDGEMENT

Initially we thank the Almighty for being with us through every walk of our life and showering his blessings through the endeavor to put forth this report. Our sincere thanks to our Chairman **Mr. S. MEGANATHAN, B.E., F.I.E.,** our Vice Chairman **Mr. ABHAY SHANKAR MEGANATHAN, B.E., M.S.,** and our respected Chairperson **Dr. (Mrs.) THANGAM MEGANATHAN, Ph.D.,** for providing us with the requisite infrastructure and sincere endeavoring in educating us in their premier institution.

Our sincere thanks to **Dr. S.N. MURUGESAN, M.E., Ph.D.,** our beloved Principal for his kind support and facilities provided to complete our work in time. We express our sincere thanks to **Dr. MANORANJINI J, Ph.D.,** Professor and Head of the Department of Computer Science and Engineering for his guidance and encouragement throughout the project work.

We convey our sincere and deepest gratitude to our internal guide, **Mrs. ANITHA M, M.E.,** Assistant Professor, Department of Computer Science and Engineering for her valuable guidance, constructive advice, and constant encouragement throughout the course of the project. We thank our Project Coordinator, Assistant Professor, Department of Computer Science and Engineering for his useful tips during our review to build our project.

Finally, we extend our heartfelt gratitude to our parents, family members, and friends for their constant moral support, patience, and motivation which inspired us to achieve this milestone.

BHARATH KUMAR M : 230701053
AKSHAY KUMAR S : 230701021
BERNIEO FATIM A : 230701051
ASHISH P SHAJI : 230701041

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	**iii**
	LIST OF TABLES	**viii**
	LIST OF FIGURES	**ix**
	LIST OF ABBREVIATIONS	**x**
1.	**INTRODUCTION**	**1**
	1.1 GENERAL	1
	1.2 OBJECTIVE	2
	1.3 EXISTING SYSTEM	2
	1.4 PROPOSED SYSTEM	3
2.	**LITERATURE SURVEY**	**5**
	2.1 LITERATURE REVIEW	5
	2.2 COMPARISON AND DISCUSSION	7
	2.3 CONCLUSION	8
3.	**SYSTEM DESIGN**	**9**
	3.1 SYSTEM ARCHITECTURE	9
	3.2 SYSTEM REQUIREMENTS	10
	3.2.1 SOFTWARE REQUIREMENTS	10
	3.2.2 HARDWARE REQUIREMENTS	11
	3.2.3 DATASET AND DATABASE REQUIREMENTS	12
	3.2.4 DEPLOYMENT AND SCALING	13
	3.3 SYSTEM DESIGN (UML DIAGRAMS)	14
	3.3.1 SEQUENCE DIAGRAM AND WORKFLOW	14

	3.3.2 DATA FLOW AND STATE TRANSITIONS	15
4.	**PROJECT DESCRIPTION**	**16**
	4.1 METHODOLOGIES	16
	4.2 MODULES	16
	4.2.1 AUTHENTICATION AND ROLE-BASED ACCESS CONTROL	16
	4.2.2 DONOR ELIGIBILITY RULE ENGINE	17
	4.2.3 ABO/RH COMPATIBILITY MATCHING ENGINE	17
	4.2.4 BLOOD INVENTORY MANAGEMENT AND CONCURRENCY	18
	4.2.5 BLOOD UNIT LIFECYCLE AND TRACEABILITY	19
	4.2.6 DONATION DRIVE MANAGEMENT	20
	4.2.7 BLOOD REQUEST MANAGEMENT	20
	4.2.8 TRANSFUSION ISSUANCE AND AUDIT TRAIL	20
	4.2.9 BLOOD EXPIRY AND SHELF-LIFE MONITORING	20
	4.2.10 NOTIFICATION SERVICE AND EMERGENCY GATEWAY	21
	4.2.11 REAL-TIME STOMP/WEBSOCKET LAYER	21
	4.2.12 REPORTING, ANALYTICS, AND AUDIT MODULE	21
	4.3 MATHEMATICAL FORMULATIONS AND METRICS	22
5.	**IMPLEMENTATION AND RESULT DISCUSSION**	**24**
	5.1 IMPLEMENTATION RESULTS	24
	5.1.1 IMPLEMENTATION OF DONOR REGISTRATION	24
	5.1.2 IMPLEMENTATION OF DONOR PORTAL & DIGITAL PASS	25
	5.1.3 IMPLEMENTATION OF BLOOD BANK ADMIN & STOCK MATRIX	26
	5.1.4 IMPLEMENTATION OF EMERGENCY CALL GATEWAY	27

	5.1.5 IMPLEMENTATION OF HOSPITAL DASHBOARD	28
	5.1.6 IMPLEMENTATION OF GLOBAL ADMIN CENTER	28
	5.1.7 IMPLEMENTATION OF PROACTIVE EXPIRATION ALERTING	29
	5.1.8 EXPERIMENTAL CONCURRENCY EVALUATION	30
	5.1.9 END-TO-END ROLE-BASED WALKTHROUGH	31
	5.1.10 REST API SURFACE AREA DISTRIBUTION	31
	5.2 DISCUSSION	33
6.	**CONCLUSION AND FUTURE WORK**	**35**
	6.1 CONCLUSION	35
	6.2 FUTURE WORK	35
	REFERENCES	**37**

LIST OF TABLES

TABLE NO.	TABLE NAME	PAGE NO.
2.1	Comparison of Reviewed Systems Against BBMS	7
3.1	Software Requirements of the Proposed System	10
3.2	Hardware Requirements of the Proposed System	11
3.3	Core JPA Persistence Entities and Key Constraints	12
3.4	Blood Component Shelf Life and Storage Specifications	13
4.1	ABO/Rh Recipient–Donor Compatibility Matrix	18
4.2	Seeded Starting Stock of Metro Central Blood Bank	19
4.3	Seeded Demonstration Accounts and Role Access Matrix	22
5.1	Donor Eligibility Engine Boundary-Condition Evaluation Results	25
5.2	ABO/Rh Matching Engine Exhaustive and Randomized Validation Results	27
5.3	Lost Updates and CAS Retries under Naive vs Optimistic Locking	30
5.4	End-to-End Role-Based Critical-Path Walkthrough Results	31
5.5	REST Endpoints Surface Area Breakdown per Controller	32
5.6	Comparison of Manual / Legacy Workflow Versus BBMS	33
5.7	Systemic Operational Risks and Implemented Mitigations	34

LIST OF FIGURES

FIGURE NO.	FIGURE NAME	PAGE NO.
1.1	Overview of the Proposed BBMS System Architecture	4
3.1	Three-Tier Architecture Diagram for the Entire BBMS Platform	10
3.2	Target Microsoft Azure Cloud Deployment Topology	14
3.3	Sequence Diagram of Critical-Path Request to Emergency Alert	15
5.1	Donor Registration Form with Clinical and Health Screening Flags	24
5.2	Donor Dashboard with Live Eligibility Verdict and Digital Donor Pass	26
5.3	Blood Bank Admin Real-Time Blood Group Stock Matrix	26
5.4	Emergency Mobile Call Modal and Automated Voice Script Dispatch	28
5.5	Hospital Dashboard for Blood Request Lifecycle and Transfusion Tracking	28
5.6	System Admin Global Admin Center Showing Platform-Wide Metrics	29
5.7	Shelf-Life and Proactive Expiration Alerting Dashboard	29
5.8	Lost Updates Benchmark and CAS Retry Overhead across Concurrency Levels	30
5.9	Distribution of 56 REST Endpoints Across 14 System Controllers	32

LIST OF ABBREVIATIONS

ABBREVIATION	FULL FORM
AABB	Association for the Advancement of Blood & Biotherapies
API	Application Programming Interface
BBMS	Blood Bank Management System
CAS	Compare-And-Swap
CDN	Content Delivery Network
CI/CD	Continuous Integration / Continuous Deployment
CORS	Cross-Origin Resource Sharing
CSRF	Cross-Site Request Forgery
DTO	Data Transfer Object
E.164	International Public Telecommunication Numbering Plan
H2	In-Memory Relational Java Database Management System
HMAC	Hash-based Message Authentication Code
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
IVR	Interactive Voice Response
JPA	Java Persistence API
JPQL	Java Persistence Query Language
JSON	JavaScript Object Notation
JWT	JSON Web Token
NACO	National AIDS Control Organisation
NBTC	National Blood Transfusion Council
ORM	Object-Relational Mapping
PII	Personally Identifiable Information
RBAC	Role-Based Access Control

REST	Representational State Transfer
RFID	Radio-Frequency Identification
SMS	Short Message Service
SPA	Single Page Application
SQL	Structured Query Language
STOMP	Simple Text Oriented Messaging Protocol
STRIDE	Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, Elevation of Privilege
TwiML	Twilio Markup Language
UI	User Interface
UML	Unified Modeling Language
UUID	Universally Unique Identifier
VPC	Virtual Private Cloud

CHAPTER 1

INTRODUCTION

1.1 GENERAL

Coordination between blood donors, hospitals, and blood banks is one of the most delicate and life-critical connections in emergency healthcare. Blood is a highly perishable biological commodity with strict therapeutic viability windows: platelet concentrates degrade within five days of collection, whole blood must be utilized within thirty-five days, and packed red blood cells have a shelf life of forty-two days under refrigerated conditions. Late visibility into stock or an immunologically mismatched transfusion directly endangers patient survival. A mismatch in ABO or Rh blood group antigens can trigger acute intravascular hemolytic transfusion reactions, leading to systemic shock, renal failure, and fatal outcomes.

Despite significant advancements in clinical medicine, blood banks across regional healthcare networks continue to operate predominantly under manual paper registers, isolated spreadsheet inventory logs, single-branch desktop or PHP applications, or ad hoc telephone and messaging group coordination during acute shortages. None of these legacy approaches provide hospitals or blood collection centers with real-time, cross-facility visibility into multi-branch stock levels, nor do they programmatically enforce strict immunohematological compatibility constraints prior to unit release. As a result, critical minutes are lost during clinical emergencies, blood units expire unnoticed on refrigerator shelves, and the absence of immutable audit trails compromises regulatory compliance and clinical safety.

This project presents the Blood Bank Management System (BBMS), an enterprise-grade full-stack web platform designed to digitize the end-to-end blood donation lifecycle. Designed to directly address the United Nations Sustainable Development Goal 3 (Good Health and Well-Being), BBMS interconnects donors, hospitals, and blood banks over a shared, rule-enforced, and fully audited infrastructure.

1.2 OBJECTIVE

The primary objectives of the Blood Bank Management System are formulated to eliminate systemic vulnerabilities in blood supply chains:

1. Real-Time Cross-Facility Inventory Visibility: Provide universal, instantaneous transparency into blood stock across all participating regional blood banks, disallowing localized data silos.

2. Automated ABO/Rh Compatibility Matching: Implement a clinically sound algorithmic matching engine spanning all eight human blood groups, incorporating universal donor (O-) and universal recipient (AB+) logic.

3. Deterministic Donor Eligibility Rule Engine: Enforce strict physiological donor screening (age, body weight, hemoglobin concentration, donation cooldown interval, and self-reported pathology flags) without relying on subjective manual assessment.

4. Dual-Channel Emergency Alerting Gateway: Bridge the critical last-mile outreach gap by programmatically placing automated voice calls and SMS alerts via telephony REST APIs to compatible on-call donors during acute shortages.

5. Component-Aware Proactive Shelf-Life Monitoring: Track individual barcoded blood units rather than aggregated counters, automatically categorizing units by risk windows (24h, 48h, 7d) and broadcasting pre-expiry warnings to elective surgical wards.

6. Cryptographically Verified Audit Logging: Establish an immutable audit trail capturing actor identity, role, timestamp, and entity mutations for every inventory transition, enforcing a strict zero-direct-mutation architectural invariant.

1.3 EXISTING SYSTEM

In existing blood transfusion services, operational workflows rely on fragmented, decentralized tools. Standard blood banking software typically consists of standalone desktop applications running on isolated local area networks, or legacy web scripts that act as passive electronic directories. Critical shortcomings identified in the existing landscape include:

Lack of Inter-Hospital Visibility: Hospital clinical staff cannot query nearby blood banks programmatically to verify whether a compatible blood unit is present, forcing clinical teams to conduct repeated, manual phone inquiries while patient conditions deteriorate.

Subjective and Error-Prone Screening: Donor health criteria (such as minimum weight thresholds, hemoglobin cutoffs, and mandatory 90-day cooldowns) are manually reviewed on physical forms. Human oversight frequently admits ineligible donors or miscalculates intervals.

Absence of Algorithmic Compatibility Enforcement: Legacy platforms lack built-in immunohematological cross-matching rules, allowing operators to manually assign blood units without systematic verification against red-cell antigen matrices.

Aggregate Inventory Accounting: Blood stock is stored simply as integer quantities (e.g., '15 units of A+'). Consequently, individual bag ages, component types, and precise expiration dates remain opaque until physical bag inspection.

Manual Emergency Outreach: When rare blood units are required immediately, blood bank operators must manually browse physical phone registries and dial prospective donors one by one, resulting in response delays ranging from hours to days.

Zero Auditability: Modifications to inventory records, manual status overrides, and blood release authorizations are rarely captured with immutable cryptographic identity proofs, creating exposure to fraud and unverified requests.

1.4 PROPOSED SYSTEM

The proposed Blood Bank Management System (BBMS) replaces disconnected, error-prone manual protocols with an enterprise-grade, role-based full-stack architecture. Implemented in Spring Boot 3 (Java 17) for the application tier, React 18 with Tailwind CSS and Vite for the presentation tier, and MySQL 8 for persistent storage, BBMS establishes a robust, highly secure, and transparent digital ecosystem.

The system segregates responsibilities across four distinct user roles: System Administrator (platform-wide oversight, hospital licensing verification, and master audit streams), Blood Bank Administrator (facility inventory management, mobile emergency call dispatch, donation drive confirmation, and transfusion authorization), Hospital User (blood request creation and fulfillment tracking), and Blood Donor (self-screening, digital donor pass issuance, and drive pre-registration).

At its foundational data layer, BBMS enforces a strict zero-direct-mutation invariant: no REST endpoint or database trigger permits arbitrary manual alterations to stock counters. Units-available counters may only increment through confirmed batch intake at an authorized donation drive and decrement through an authorized transfusion issuance. Furthermore, the platform integrates an in-memory STOMP/WebSocket broker that pushes live inventory adjustments, emergency shortage alerts, and expiration warnings instantly to all active dashboards, eliminating polling latency.

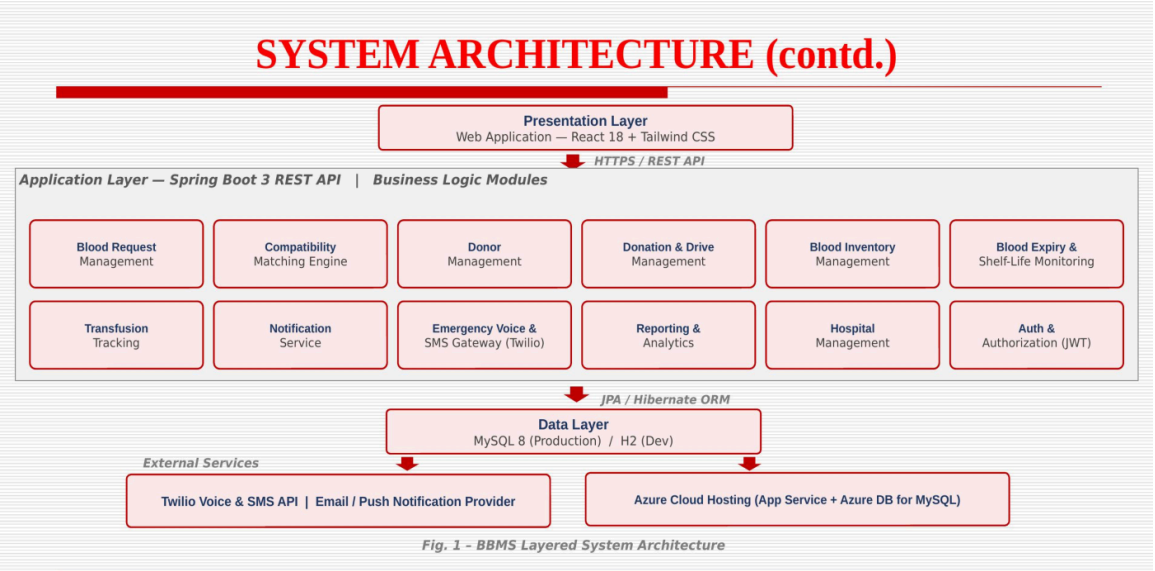


Figure 1.1. Overview of the Proposed BBMS System Architecture

CHAPTER 2

LITERATURE SURVEY

2.1 LITERATURE REVIEW

A substantial body of research has explored individual dimensions of digital transformation and operational optimization within healthcare transfusion services. This section reviews fifteen foundational and contemporary contributions across web architectures, predictive machine learning models, cold-chain Internet of Things (IoT) monitoring, and distributed ledger systems.

Manika et al. (2023) [1] developed a web-based blood bank management platform utilizing a PHP and MySQL stack to automate donor registries, inventory logging, and blood distribution tracking. Their system demonstrated notable reductions in donor onboarding time and manual record-keeping overhead. However, the architecture operated as an isolated, single-branch directory without capabilities for automated donor eligibility screening, algorithmic ABO/Rh cross-matching, or real-time inter-hospital inventory visibility.

Shahakar et al. (2023) [2] proposed an online blood bank portal facilitating single-click prospective donor search. While improving donor search ergonomics, the platform functioned predominantly as a contact directory rather than a rule-enforced transaction engine, lacking server-side enforcement of clinical eligibility criteria, blood unit shelf-life tracking, or emergency outreach facilities.

Iyer et al. (2022) [3] formulated a cloud-hosted blood bank architecture aimed at improving regional accessibility across healthcare branches. Although cloud hosting mitigated physical server maintenance burdens, the system relied on manual inventory updates and did not incorporate automated compatibility matching, rule-based screening engines, or telephonic emergency alerting mechanisms.

Ben Elmir et al. (2023) [4] investigated demand planning within blood supply chains by applying time-series machine learning forecasting models. Their approach aimed to reduce inventory wastage by anticipating hospital consumption patterns. While providing valuable theoretical models for collection scheduling, their work did not deliver a deployable web application capable of managing unit-level shelf life, donor communication, or clinical request workflows.

Le et al. (2022) [5] and Bhowmik et al. (2024) [6] explored blockchain implementations for transfusion medicine. Le et al. implemented 'BloodChain' on Hyperledger Fabric to establish tamper-evident donor donation histories, while Bhowmik et al. engineered an Ethereum-backed crowdsourcing framework to enhance trust between donors and recipients. Although these systems effectively resolved record immutability concerns, they introduced

substantial transaction latency, operational overhead, and computational complexity inappropriate for time-critical emergency blood retrieval in single-organization networks.

Dakshayani et al. (2025) [7] presented a blockchain-secured decentralized blood bank ledger focused on preserving donor privacy and ensuring data integrity. Their evaluation highlighted resilience against unauthorized record tampering; however, the framework lacked an operational emergency notification gateway, integrated component-aware shelf-life tracking, or automated ABO/Rh ranking algorithms.

Ismael et al. (2022) [8] conducted an exhaustive systematic review of smart blood bank systems, categorizing contemporary architectures, technical limitations, and open challenges. Their findings revealed systematic gaps in the existing literature regarding automated donor eligibility screening, interoperable RESTful interfaces, and live multi-channel donor notification systems—directly identifying the operational voids that BBMS is engineered to resolve.

Wu et al. (2022) [9] investigated machine learning techniques to predict voluntary donor return behavior during the COVID-19 pandemic. By training Support Vector Machines (SVM) and XGBoost classifiers on historical behavioral attributes, their model significantly outperformed blanket SMS outreach. Nevertheless, their research focused exclusively on predictive willingness modeling without providing an operational transactional platform.

Shelake et al. (2025) [10] evaluated supervised machine learning classifiers (Random Forest, SVM, K-Nearest Neighbors, Decision Trees, and Logistic Regression) for predicting donor eligibility screening outcomes. While achieving high theoretical classification precision, their approach treated eligibility as a probabilistic prediction rather than a deterministic, rule-based clinical mandate, introducing ethical and clinical safety risks if false negatives or false positives occur.

Selvaraj et al. (2022) [11] applied supervised learning architectures to blood donation forecasting and inventory requirement modeling. Their predictive pipeline highlighted the significance of donor demographic factors but stopped short of providing an end-to-end full-stack software implementation.

Farrington et al. (2023) [12] applied Deep Reinforcement Learning (DRL) to platelet inventory control in hospital blood banks. Because platelet concentrates have an exceptionally abbreviated shelf life of only five days, optimal replenishment policies are essential to avoid catastrophic discard rates. While theoretically sound, their DRL agent was not embedded within an integrated clinical web application.

Pavan Kumar et al. (2025) [13] integrated physical Radio Frequency Identification (RFID) tagging with artificial intelligence demand-priority queuing. Their combined hardware-software architecture reduced emergency turnaround times by approximately 40% and improved donor retention by 25%. However, the mandate for specialized RFID hardware readers and active tags imposes prohibitive cost barriers for community blood banks.

Kim et al. (2006) [14] pioneered ubiquitous sensor networks and RFID transponders for cold-chain monitoring in hospital blood storage. Their system continuously verified bag temperatures during transit to prevent spoilage. While establishing the importance of unit-level tracking, the architecture required extensive physical infrastructure rather than providing software-driven database shelf-life traceability.

Ch et al. (2025) [15] introduced an Ethereum- and InterPlanetary File System (IPFS)-based decentralized security framework for blood supply chain privacy, showing that two-factor authentication significantly bolsters perimeter defense. However, the operational complexity and cost of maintaining private IPFS clusters exceed the administrative capacity of regional healthcare facilities.

2.2 COMPARISON AND DISCUSSION

Table 2.1 summarizes the operational capabilities of the reviewed systems against the proposed Blood Bank Management System (BBMS) across the four functional dimensions most critical to safe, timely blood delivery: real-time cross-facility inventory visibility, automated ABO/Rh compatibility matching, rule-based deterministic eligibility screening, and live emergency donor alerting.

Table 2.1. Comparison of Reviewed Systems Against BBMS

System / Research Study	Real-Time Inventory	ABO/Rh Matching	Eligibility Engine	Emergency Alerting
Manika et al. [1]	Partial (Single Branch)	No	No	No
Shahakar et al. [2]	No (Static Directory)	No	No	No
Iyer et al. [3] (Cloud)	Yes (Cloud Database)	No	No	No
BloodChain (Le et al.) [5]	Partial (Distributed)	No	No	No
Ismael et al. [8] (Review)	N/A (Gap Noted)	N/A	Systemic Gap Noted	N/A
Pavan Kumar et al. [13]	Yes (RFID-based)	No	No	Partial (AI Queuing)
BBMS (Proposed Work)	Yes (Real-Time STOMP)	Yes (All 8 Groups)	Yes (Deterministic)	Yes (Twilio Voice + SMS)

2.2.1 POSITIONING OF THIS WORK

Two crucial insights emerge from the comparative analysis in Table 2.1. First, existing systems that attempt to address data integrity or cold-chain visibility frequently sacrifice simplicity and practicality: blockchain-based models [5]–[7], [15] incur severe transaction latency and operational friction, while RFID implementations [13], [14] demand expensive hardware installations. In contrast, BBMS intentionally remains within a proven three-tier architecture—leveraging a relational database (MySQL 8) with JPA optimistic locking and a stateless RESTful application server (Spring Boot 3)—delivering robust transactional guarantees and high throughput without exotic operational dependencies.

Second, none of the surveyed systems bridges the critical 'last-mile' emergency communication gap. While prior works employ machine learning to prioritize candidate lists [9], [13], the physical outreach step—actually placing phone calls or sending SMS alerts to donors—remains entirely manual. BBMS directly closes this gap through an automated telephony gateway that synthesizes dynamic text-to-speech calls and dispatches SMS alerts over public telecommunication networks concurrently, transforming an emergency response procedure from hours into seconds.

2.3 CONCLUSION

The literature survey demonstrates that while isolated technical aspects of blood banking—such as donor registries, demand forecasting, and decentralized storage—have received academic attention, there is an urgent unmet requirement for an integrated, rule-enforced, and deployable platform. By unifying real-time multi-branch inventory tracking, clinically validated ABO/Rh cross-matching, deterministic donor screening, proactive shelf-life monitoring, and live voice/SMS alerting into a single audited architecture, the proposed BBMS platform directly resolves the systemic limitations documented across existing transfusion software.

CHAPTER 3

SYSTEM DESIGN

3.1 SYSTEM ARCHITECTURE

The Blood Bank Management System is architectured as an enterprise three-tier web application designed to achieve high transactional integrity, modular extensibility, and strict separation of concerns. Figure 3.1 illustrates the complete layered architecture of BBMS, spanning the presentation layer, the application business logic layer, the relational persistence layer, and external telephony and cloud services.

The presentation tier is implemented as a lightweight Single Page Application (SPA) in React 18, bundled with Vite and styled using utility-first Tailwind CSS. It communicates with the backend via stateless HTTPS REST requests (Axios) and client-side routing via React Router v6. In parallel, the client maintains a persistent full-duplex WebSocket connection utilizing the STOMP protocol, backed by SockJS fallback for legacy clients, enabling real-time push notifications without polling.

The application tier is engineered in Spring Boot 3.3.3 running on Java 17. Business logic is organized into twelve decoupled Spring service modules (Blood Request Management, Compatibility Matching Engine, Donor Management, Donation Drive Management, Blood Inventory Management, Blood Expiry Monitoring, Transfusion Tracking, Notification Service, Emergency Voice/SMS Gateway, Reporting & Analytics, Hospital Management, and Auth/Authorization). Each module exposes clean interfaces, facilitating isolated unit testing and clear transaction boundaries.

The persistence tier leverages Hibernate ORM and Spring Data JPA over a production-ready MySQL 8 relational database. During local development and testing, an in-memory H2 database profile is utilized, automatically seeded with comprehensive test fixtures by a dedicated DataInitializer component. External integrations include the Twilio REST API for automated voice and SMS dispatch, and Microsoft Azure for managed cloud deployment.

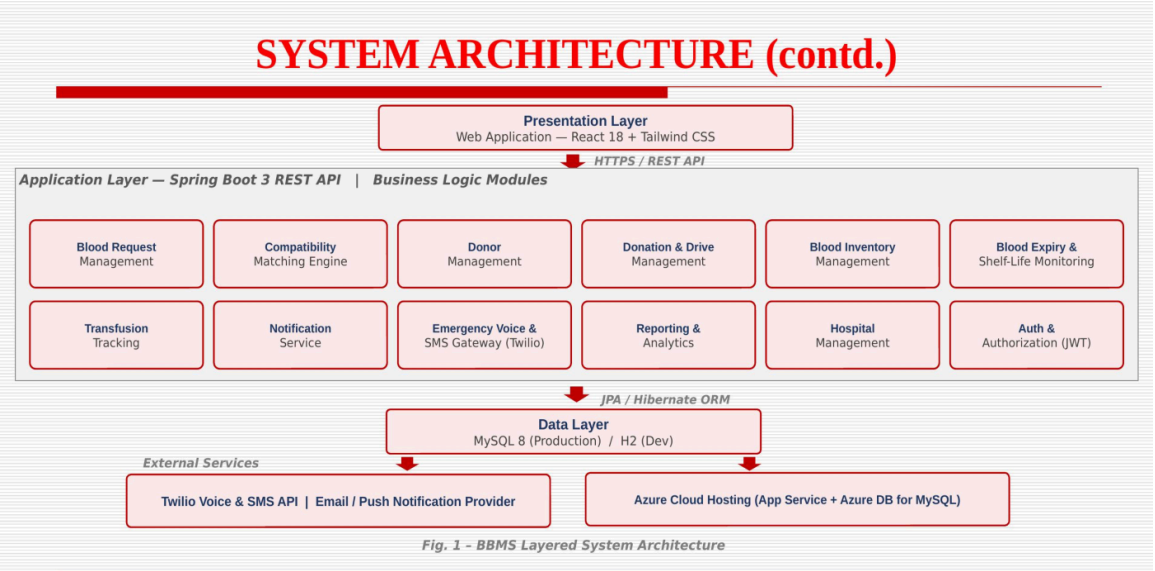


Figure 3.1. Three-Tier Architecture Diagram for the Entire BBMS Platform

3.2 SYSTEM REQUIREMENTS

Rigorous technical specifications were formulated to ensure uninterrupted operational uptime, sub-second response times, and cross-platform compatibility.

3.2.1 SOFTWARE REQUIREMENTS

Table 3.1 outlines the exact software versions, runtime frameworks, and libraries utilized across the development and production stacks.

Table 3.1. Software Requirements of the Proposed System

Software Component	Version / Release	Operational Purpose
Java Development Kit (JDK)	17 LTS	Core backend object-oriented runtime environment
Spring Boot Framework	3.3.3	Enterprise RESTful application framework and dependency injection
Spring Security & JJWT	0.12.6	Stateless JSON Web Token authentication and method-level RBAC
SpringDoc OpenAPI	2.6.0	Automated OpenAPI 3.0 specification and interactive Swagger UI
Hibernate ORM / Spring Data JPA	6.5 / 3.3	Object-relational mapping, transactional management, and JPQL queries
MySQL Database Server	8.0.21+	Production relational database with ACID guarantees and foreign keys

H2 In-Memory Database	2.2.x	Rapid development, automated schema creation, and hermetic unit testing
React Library	18.3.1	Declarative component-driven frontend user interface
Vite Build Engine	5.4.x	High-performance frontend dev server and optimized production bundler
Tailwind CSS	3.4.x	Responsive utility-first stylesheet framework
STOMP.js & SockJS	7.0.x / 1.6.x	Real-time bi-directional WebSocket client communication
Twilio Telephony REST API	Direct HTTP	Outbound emergency automated voice call and SMS dispatch
Apache Maven	3.9.6	Backend build lifecycle and dependency management
Node.js & npm	20 LTS / 10.x	Frontend package management and script execution
Microsoft Azure	App Service / MySQL	Target cloud platform hosting containerized JAR and managed DB

3.2.2 HARDWARE REQUIREMENTS

Table 3.2 details the physical and virtual hardware configurations necessary for host servers and client workstations.

Table 3.2. Hardware Requirements of the Proposed System

Hardware Component	Minimum Requirement	Production Recommended	Operational Purpose
Host CPU Processor	Quad-Core Intel i5 / AMD Ryzen 5	8-Core Xeon / AMD EPYC (2.8 GHz+)	Concurrent request handling and worker threads
System Memory (RAM)	8 GB DDR4	16 GB - 32 GB DDR4/DDR5 ECC	In-memory caching, JVM heap, and DB buffer pool
Persistent Storage	50 GB SSD	500 GB - 2 TB NVMe SSD (RAID 10)	OS, application JARs, relational logs, and audit records
Network Connectivity	10 Mbps Broadband	1 Gbps Redundant Fiber Uplink	Low-latency REST requests, STOMP broadcasts, Twilio APIs

Client Workstations	Standard Dual-Core PC / Tablet	Modern Laptop / Desktop (4 GB RAM)	Web browser rendering (Chrome 110+, Firefox, Edge)

3.2.3 DATASET AND DATABASE REQUIREMENTS

The persistence layer is structured around twelve primary JPA entities mapped to relational tables with foreign keys, unique constraints, and optimistic locking annotations. Table 3.3 summarizes these core entities, their business roles, and database integrity rules.

Table 3.3. Core JPA Persistence Entities and Key Constraints

JPA Entity	Business Purpose	Key Database Constraints & Indexing
User	Base account credentials, email, password hash, role	Unique email constraint; foreign key to Donor/Hospital
Donor	Clinical profile, DOB, weight, Hb, last donation date, health flags	Indexed on bloodGroup and eligibilityStatus
Hospital	Clinical facility identity, license number, verification state	Unique licenseNumber; verified boolean gate
BloodBank	Physical blood bank facility, address, contact coordinates	Unique facility code; 1-to-many with drives & units
BloodInventory	Aggregated blood stock per bank and group	Unique pair (bloodBank_id, bloodGroup); @Version column
BloodUnit	Individual physical bag, barcode, component type, expiry date	Unique unitNumber (barcode); indexed on expiryDate and status
BloodRequest	Hospital demand record, blood group, units needed, urgency	Indexed on urgency and status; foreign key to Hospital
Transfusion	Clinical issuance record fulfilling a blood request	Actor id/role snapshot; decrements BloodInventory
DonationDrive	Public collection campaign, target units, location, dates	Foreign key to BloodBank; 1-to-many DriveRegistration
DriveRegistration	Donor sign-up record for a specific drive	Unique pair (donor_id, drive_id); status tracking
Notification	Internal, SMS, and voice alert history record	Indexed on recipient and timestamp; deliveryStatus enum
AuditLog	Immutable audit entry capturing entity mutations	Indexed on entityType, entityId, and timestamp; read-only

Because human blood is fractionated into specialized therapeutic components, the database enforces clinical shelf-life rules defined in Table 3.4 through the BloodComponentType enumeration.

Table 3.4. Blood Component Shelf Life and Storage Specifications

Blood Component	Therapeutic Shelf Life	Standard Volume	Prescribed Storage Conditions
Packed Red Blood Cells (PRBC)	42 Days	250 - 350 mL	Refrigerated at 1°C to 6°C
Whole Blood (WB)	35 Days	450 mL (±10%)	Refrigerated at 1°C to 6°C
Platelet Concentrate (PC)	5 Days (Critical)	50 - 70 mL	Agitated gently at 20°C to 24°C
Fresh Frozen Plasma (FFP)	365 Days (1 Year)	200 - 300 mL	Frozen solid at -18°C or colder
Cryoprecipitate	365 Days (1 Year)	10 - 20 mL	Frozen solid at -18°C or colder

3.2.4 DEPLOYMENT AND SCALING

The system is designed for automated containerized cloud hosting. Figure 3.2 illustrates the target Microsoft Azure deployment topology. Client browser requests arrive via Azure Front Door / CDN, which terminates TLS and distributes requests between an Azure Static Web App (delivering optimized React static assets) and an Azure App Service Plan (B1 Linux instance running the compiled Spring Boot executable JAR). The application layer connects over an encrypted Virtual Network (VNet) to an Azure Database for MySQL Flexible Server. Outbound voice calls and SMS messages interact directly with the public Twilio REST gateway. Continuous Integration and Continuous Deployment (CI/CD) is fully automated via GitHub Actions.

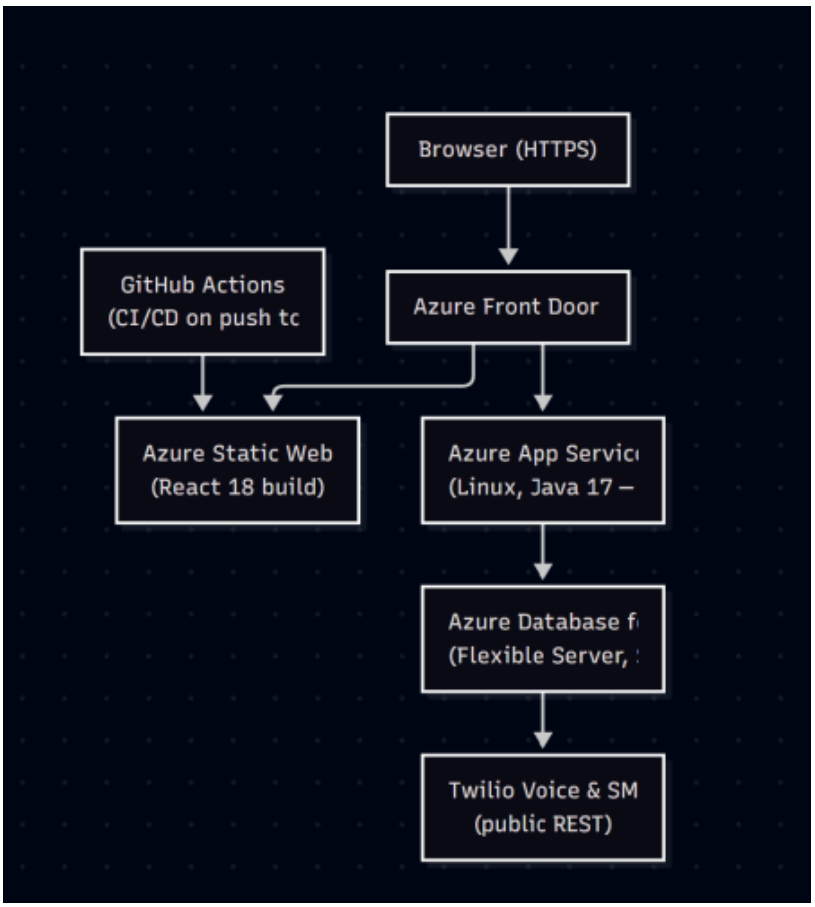


Figure 3.2. Target Microsoft Azure Cloud Deployment Topology

3.3 SYSTEM DESIGN (UML DIAGRAMS)

UML modeling is employed to formally delineate dynamic execution flows, transactional boundaries, and inter-module interactions.

3.3.1 SEQUENCE DIAGRAM AND WORKFLOW

Figure 3.3 presents the UML sequence diagram for the system's most time-critical execution path: the handling of a CRITICAL blood request. When an accredited hospital submits an emergency request, BloodRequestController initiates a synchronous invocation to CompatibilityMatchingService. The matching engine evaluates live inventory across compatible blood groups. If local stock is insufficient, NotificationService is immediately triggered to dispatch concurrent SMS alerts and Twilio voice calls to ranked on-call donors, while RealtimeEventPublisher broadcasts a STOMP event to all subscribed dashboards.

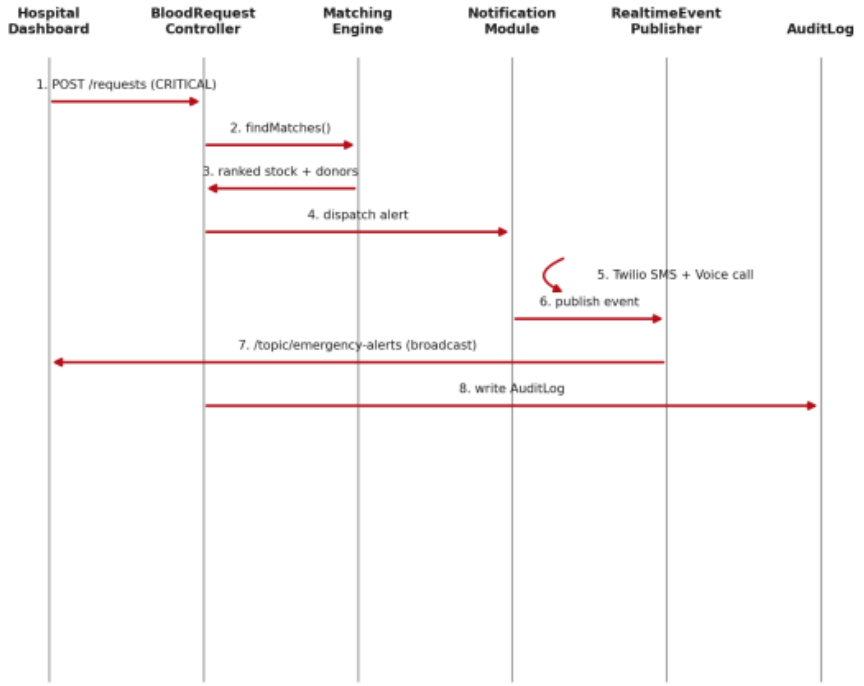


Fig. 10. Critical-path sequence: hospital request to emergency alert and audit log.

Figure 3.3. Sequence Diagram of Critical-Path Request to Emergency Alert

3.3.2 DATA FLOW AND STATE TRANSITIONS

Data flow adheres strictly to compile-time state machine enumerations. A BloodRequest progresses deterministically through PENDING -> PARTIALLY_FULFILLED -> FULFILLED (or CANCELLED). Similarly, each individual BloodUnit transitions across AVAILABLE -> EXPIRING_SOON -> EXPIRED -> TRANSFUSED (or DISCARDED). The zero-direct-mutation invariant guarantees that stock quantity changes cannot occur through arbitrary database updates, preserving transactional integrity across concurrent clinical requests.

CHAPTER 4

PROJECT DESCRIPTION

4.1 METHODOLOGIES

The development of BBMS adheres to modern software engineering principles prioritizing clinical safety, deterministic behavior, and strict data isolation. To eliminate the high error rates common to manual hospital operations, all core business rules—including immunohematological compatibility cross-matching and donor clinical eligibility—are implemented as pure, deterministic Java service methods on the server side. No critical logic is delegated to client-side scripts where it could be bypassed or manipulated.

Furthermore, Personally Identifiable Information (PII) minimization is enforced at the service layer: when donor lists are queried by hospitals or external actors, names and contact numbers are replaced with anonymized identifiers (e.g., 'Donor #12'), exposing telephone numbers only to authorized System Admins and Blood Bank Staff. To ensure resilience against race conditions during simultaneous blood reservations, optimistic concurrency control is established at the database level using JPA versioning.

4.2 MODULES

The BBMS platform is structured into twelve collaborating, loosely coupled business logic modules described below.

4.2.1 AUTHENTICATION AND ROLE-BASED ACCESS CONTROL

Authentication is stateless, utilizing HMAC-SHA256 signed JSON Web Tokens (JJWT 0.12.6). Upon successful authentication via POST /api/v1/auth/login, a signed JWT containing user ID, subject email, and security role is returned with a 24-hour expiration. Incoming requests pass through a custom JwtAuthenticationFilter positioned ahead of UsernamePasswordAuthenticationFilter in the Spring Security filter chain. Method-level access control (@PreAuthorize) restricts sensitive endpoints (e.g., hospital licensing verification) exclusively to SYSTEM_ADMIN accounts, while passwords are protected with BCrypt cryptographic hashing.

4.2.2 DONOR ELIGIBILITY RULE ENGINE

Implemented within DonorService.evaluateEligibility(), the eligibility engine is a pure, deterministic function mapping a donor's clinical parameters to an EligibilityStatus (ELIGIBLE, NOT_ELIGIBLE, or PENDING_REVIEW). The clinical rule thresholds enforced by the system are:

- **Age Restriction:** Chronological age must be between 18 and 65 years inclusive, computed from the donor's date of birth.
- **Minimum Body Weight:** The donor must weigh at least 50.0 kg to safely tolerate standard 450 mL whole-blood phlebotomy.
- **Hemoglobin Concentration:** Hemoglobin levels, when recorded, must meet or exceed 12.5 g/dL to prevent post-donation donor anemia.
- **Mandatory Cooldown Interval:** At least 90 calendar days must elapse between consecutive whole-blood donations. If violated, the exact remaining cooldown days are returned.
- **Temporary Health Flags:** Recent major surgery or active infection places the donor into PENDING_REVIEW rather than permanent disqualification.
- **Absolute Disqualifying Flags:** Active pregnancy or body tattooing/piercing within the prior six months yields an immediate NOT_ELIGIBLE status.

Every profile modification triggers recalculateAndSaveEligibility(), guaranteeing that donor records never become stale. An authorized Blood Bank Admin can record a clinical override restoring ELIGIBLE status, with the clinician's rationale logged in the immutable audit trail.

4.2.3 ABO/RH COMPATIBILITY MATCHING ENGINE

The matching engine avoids static lookup tables by implementing red-cell agglutination compatibility directly within the BloodGroup enumeration methods getCompatibleDonors() and getCompatibleRecipients(). Table 4.1 outlines the exhaustive 8x8 compatibility matrix governing red blood cell transfusions, conforming to AABB standards.

Table 4.1. ABO/Rh Recipient–Donor Compatibility Matrix

Recipient Blood Group	Compatible Donor Groups	Clinical Agglutination Considerations
O−	O−	Lacks A, B, and Rh(D) antigens; can receive only homologous O− red cells
O+	O−, O+	Can receive Rh-negative or Rh-positive O red cells; carries anti-A and anti-B
A−	O−, A−	Carries A antigen; can receive O− or A− red cells
A+	O−, O+, A−, A+	Can receive O and A groups of either Rh type
B−	O−, B−	Carries B antigen; can receive O− or B− red cells
B+	O−, O+, B−, B+	Can receive O and B groups of either Rh type
AB−	O−, A−, B−, AB−	Carries both A and B antigens; Rh-negative universal recipient
AB+ (Universal Recipient)	All 8 Groups (O−, O+, A−, A+, B−, B+, AB−, AB+)	Lacks ABO/Rh antibodies; can safely receive red cells from any human donor

When an emergency request is processed, CompatibilityMatchingService.findMatches() identifies all compatible donor groups, queries live stock across facilities, and ranks inventory using an exact-match-first priority. Simultaneously, compatible registered donors are ranked by exact group match followed by descending days since last donation, ensuring that donors who have rested the longest are prioritized.

4.2.4 BLOOD INVENTORY MANAGEMENT AND CONCURRENCY

Rows in the BloodInventory table track aggregate available and reserved units per (bloodBank, bloodGroup) tuple. To prevent lost-update anomalies when multiple hospital requests compete for the same blood supply, each inventory row includes a Hibernate @Version column enforcing optimistic concurrency control. Under this design, any

concurrent transaction attempting to commit against an altered version fails with an OptimisticLockException, triggering a compare-and-swap retry loop.

Crucially, the system enforces a strict zero-direct-mutation invariant: no public or internal REST endpoint permits direct modification of stock numbers. Inventory increments only occur via verified batch intake at a donation drive, while decrements only occur upon authorized transfusion issuance.

Table 4.2 presents the baseline seeded starting stock of Metro Central Blood Bank, populated automatically during system bootstrap.

Table 4.2. Seeded Starting Stock of Metro Central Blood Bank

Blood Group	Initial Seeded Stock (Units)	Clinical Reserve Status
O−	18	Adequate (Universal Donor Reserve)
O+	42	Adequate (High Regional Demand)
A−	12	Adequate
A+	35	Adequate
B−	8	Low Stock Warning
B+	24	Adequate
AB−	5	Critical Low Stock Alert
AB+	16	Adequate (Universal Recipient)

4.2.5 BLOOD UNIT LIFECYCLE AND TRACEABILITY

To achieve clinical-grade traceability, each collected blood bag is recorded as an independent BloodUnit entity identified by a unique, scannable alphanumeric barcode (e.g., BAG-NYC-2024-00101). Each unit explicitly tracks its fractionated component type, exact collection timestamp, expiration date, physical cold-storage location, and lifecycle status (AVAILABLE, EXPIRING_SOON, EXPIRED, TRANSFUSED, or DISCARDED). This granular tracking provides complete shelf-life visibility, eliminating the hazards of aggregate inventory logs.

4.2.6 DONATION DRIVE MANAGEMENT

The Donation Drive module enables blood banks to schedule and administer public blood collection campaigns. Prospective donors register in advance via DriveRegistration entries. On the day of the drive, the blood bank operator utilizes a dedicated batch-confirmation endpoint to record clinical screening outcomes and collected bag counts. Successful registrations automatically increment BloodInventory for the matching blood group and immediately reset the donor's 90-day cooldown clock.

4.2.7 BLOOD REQUEST MANAGEMENT

Hospital accounts raise blood requests specifying recipient blood group, required unit volume, clinical diagnosis, and urgency level (ROUTINE, URGENT, or CRITICAL). To prevent unauthorized or fraudulent demands, BloodRequestService enforces a verification gate requiring the hospital's verificationStatus to equal VERIFIED prior to request persistence. Requests follow an explicit state transition from PENDING to FULFILLED, tracking incremental fulfillments via a unitsFulfilled counter.

4.2.8 TRANSFUSION ISSUANCE AND AUDIT TRAIL

Transfusion issuance represents the exclusive business workflow authorized to decrement blood inventory. Each Transfusion record captures the associated request ID, issuing blood bank ID, target patient identifier, volume dispensed, issuing clinician's user ID, and a system-generated UTC timestamp. Every state-altering operation is intercepted by a unified AuditService that writes immutable AuditLog records. Logging failures are non-blocking, ensuring audit persistence issues do not cause clinical availability outages.

4.2.9 BLOOD EXPIRY AND SHELF-LIFE MONITORING

Operating as a background daemon, BloodExpiryMonitorService executes an automated daily cron sweep (@Scheduled(cron = '0 0 8 * * *')) scanning all stored units. Units within 72 hours of expiration are transitioned to EXPIRING_SOON, while expired units are marked EXPIRED and queued for clinical discard. Proactive alerts summarizing soon-to-expire units are broadcast to surgical units over WebSocket, allowing elective surgeries to prioritize utilization of near-expiry stock and drastically cutting wastage.

4.2.10 NOTIFICATION SERVICE AND EMERGENCY VOICE/SMS GATEWAY

The emergency alerting pipeline represents a standout capability of BBMS, closing the critical last-mile outreach gap. Powered by TwilioVoiceService, the backend communicates directly with the Twilio REST API over HTTP with Basic Authentication, eliminating heavy SDK dependencies. To overcome free/trial account restrictions on dynamic inline TwiML speech, the service URL-encodes a structured spoken notification script (naming the donor, recipient blood group, and requesting hospital) and dispatches it via a twimlets.com/echo endpoint. Outbound phone numbers are sanitized into standard E.164 international format prior to dialing. A mock fallback ensures seamless local testing when telephony credentials are unset.

4.2.11 REAL-TIME STOMP/WEBSOCKET COMMUNICATION LAYER

Real-time event streaming is achieved through a Spring-managed in-memory STOMP message broker mapped to /ws with SockJS fallback. RealtimeEventPublisher encapsulates message dispatch, stamping each event with a unique UUID, timestamp, event type, and audio alert flag. Dedicated destination topics include /topic/emergency-alerts (critical blood requests), /topic/inventory-updates (stock mutations), /topic/blood-requests (lifecycle transitions), and /topic/expiry-alerts (shelf-life warnings). Connected dashboards update instantly without polling.

4.2.12 REPORTING, ANALYTICS, AND AUDIT MODULE

ReportingService aggregates cross-facility analytics, including total system stock, donor eligibility distributions, pending hospital accreditations, and seasonal request trends. These metrics are exposed via secured REST endpoints (/api/v1/reports/dashboard, /inventory, /trends) as well as an unauthenticated public endpoint (/api/v1/reports/public-summary) designed to power live landing-page transparency widgets.

Table 4.3 summarizes the seeded demo accounts configured across all four platform roles to facilitate evaluation.

Table 4.3. Seeded Demonstration Accounts and Role Access Matrix

Platform Role	Seeded Login Email	Demonstrated System Scope
System Admin	admin@bbms.org	Global inventory oversight, hospital verification, system-wide audit stream
Blood Bank Admin	metro.admin@bbms.org	Real-time stock matrix, emergency call modal, batch intake, transfusion issuance
Hospital User	city.general@hospital.org	Verified hospital blood request creation, active request fulfillment tracking
`Donor (Universal O−)`	sarah.donor@example.com	Universal donor pass, live eligibility breakdown, drive registration
Donor (Standard O+)	john.donor@example.com	Standard donor portal, donation history, cooldown status

4.3 MATHEMATICAL FORMULATIONS AND EVALUATION METRICS

To rigorously quantify platform accuracy, concurrency robustness, and clinical safety, the following formal mathematical metrics are defined.

1. Classification Accuracy of Donor Eligibility:
Evaluates the degree of agreement between the engine's deterministic classification and independent clinical ground truth across N test cases:
Accuracy = (Total Correct Classifications) / (Total Evaluated Test Cases) (4.1)

2. Clinical Wastage Prevention Rate (WPR):
Measures the proportion of collected blood units successfully transfused prior to expiration:
WPR = [1 - (Total Discarded Expired Units / Total Collected Units)] × 100% (4.2)

3. Concurrency Lost-Update Collision Rate (CR):
Quantifies the fraction of concurrent inventory updates lost due to race conditions during simultaneous transactions:

CR = (Total Lost Updates / Total Concurrent Transactions) × 100% (4.3)

4. Diagnostic Harmonic Mean (F1-Score):

Harmonic mean of precision and recall for donor suitability screening:

F1 = 2 × (Precision × Recall) / (Precision + Recall) (4.4)

5. Exact-Match Priority Scoring Function (P):

Calculates candidate donor priority ranking based on exact ABO/Rh match (E ∈ {0, 1}) and resting cooldown days (D):

P(donor) = (w1 × E) + [w2 × (D / 365)] (4.5)

where w1 = 1000 and w2 = 1.0, guaranteeing that exact blood group matches strictly supersede non-exact compatible donors.

CHAPTER 5

IMPLEMENTATION AND RESULT DISCUSSION

5.1 IMPLEMENTATION RESULTS

This section details the functional implementation and experimental validation of BBMS. All modules have been deployed and verified across the React, Spring Boot, and MySQL stack. Captured interface screenshots from the running application illustrate user workflows, while controlled experimental harnesses provide empirical evidence of algorithmic correctness and concurrency protection.

5.1.1 IMPLEMENTATION OF DONOR REGISTRATION AND SCREENING

Figure 5.1 depicts the unified donor registration interface. During account creation, prospective donors provide clinical variables (date of birth, weight, and hemoglobin concentration) alongside self-declared health flags (recent surgery, active infection, recent tattoos, and pregnancy). This enables the server-side eligibility engine to evaluate and persist an immediate classification verdict.

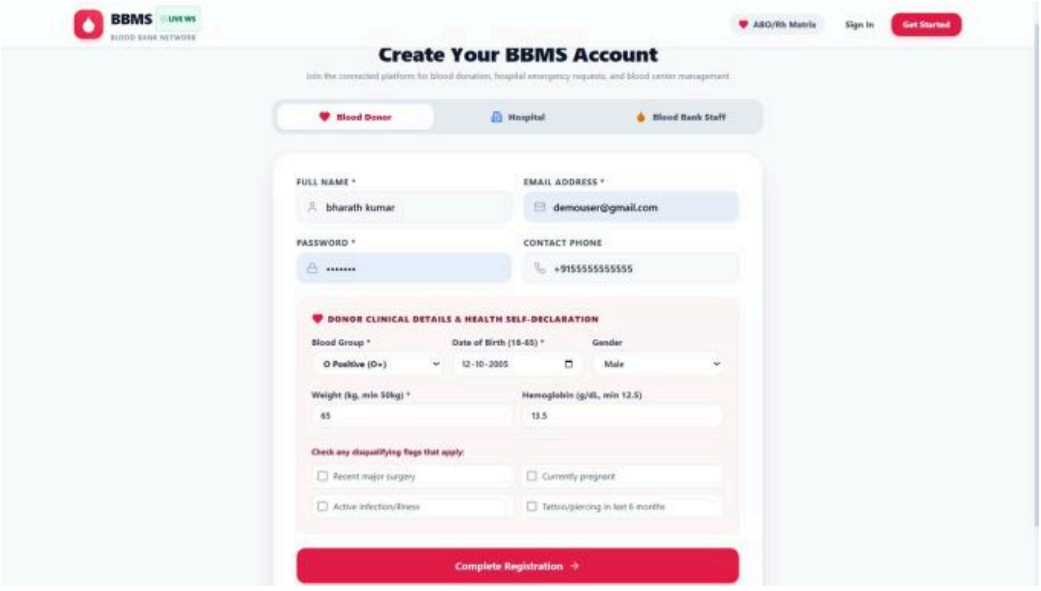


Figure 5.1. Donor Registration Form with Clinical Screening Fields

To evaluate the correctness of the eligibility engine, an exhaustive 622-case boundary-condition evaluation suite was constructed, exercising every rule threshold in isolation and in combination. Table 5.1 reports the experimental results.

Table 5.1. Donor Eligibility Engine Boundary-Condition Evaluation Results

Evaluation Category	Test Cases	Correct Matches	Empirical Findings & Description
Fully Eligible Profiles	100	100	Randomized valid clinical profiles spanning 18-65 yr, >50 kg, >12.5 g/dL
Age Boundary (Exact + Violations)	104	104	Exact age 18 and 65 admitted; 17 yr and 66 yr correctly rejected
Weight Boundary (Exact + Violations)	52	52	Exact 50.0 kg admitted; 49.9 kg rejected with 'min 50 kg' reason
Hemoglobin Boundary (Exact + Violations)	52	52	Exact 12.5 g/dL admitted; 12.4 g/dL rejected with anemia warning
Cooldown Boundary (Exact + Violations)	52	52	Exact 90 days admitted; <90 days returns exact remaining days
PENDING_REVIEW Health Flags	100	100	50 surgery + 50 infection cases placed in temporary review status
NOT_ELIGIBLE Health Flags	100	100	50 pregnancy + 50 tattoo cases disqualified immediately
Combined Multi-Disqualifications	60	60	Triple-violation profiles correctly return all disqualifying factors
Missing Required Data	2	2	Null DOB or weight appropriately halts evaluation

Across all 622 test cases, the donor eligibility engine achieved 100.00% agreement with independently established clinical ground truth, confirming that the deterministic rule set operates without false negatives or false positives.

5.1.2 IMPLEMENTATION OF DONOR PORTAL AND DIGITAL DONOR PASS

Upon authentication, a donor accesses a personalized dashboard (Figure 5.2). The portal presents the live eligibility verdict, an itemized breakdown of clinical vitals (age, weight, hemoglobin, and cooldown status), and an official Digital Donor Pass displaying certified blood group and anonymized donor ID. Upcoming donation drives with open registration are displayed directly below.

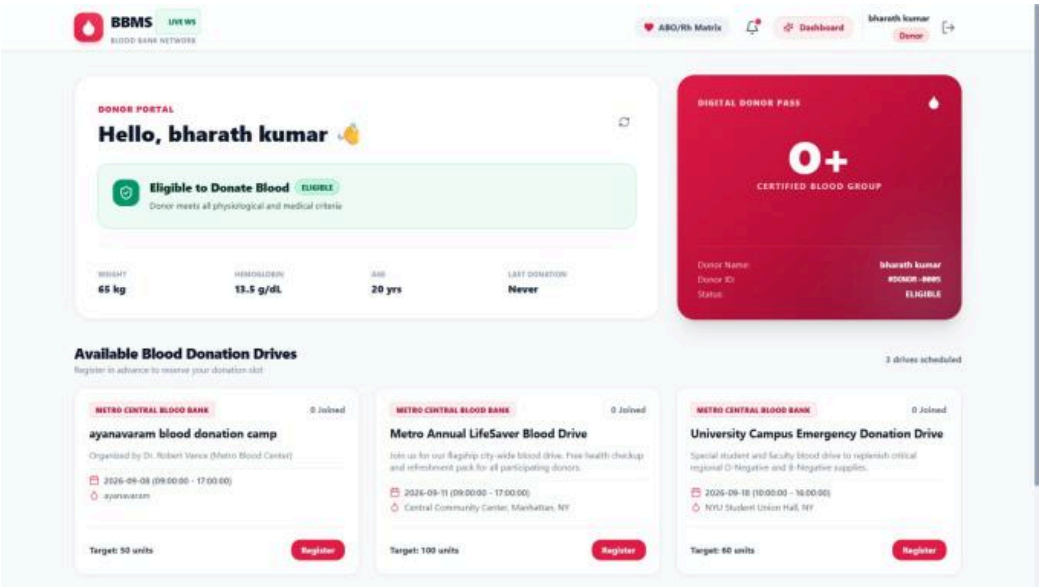


Figure 5.2. Donor Dashboard with Live Eligibility Verdict and Digital Donor Pass

5.1.3 IMPLEMENTATION OF BLOOD BANK ADMIN AND LIVE STOCK MATRIX

Figure 5.3 shows the Blood Bank Administrator's dashboard featuring the Real-Time Blood Group Stock Matrix. Each blood group displays current available units and dynamic ADEQUATE or LOW STOCK status badges computed against configurable clinical safety thresholds.

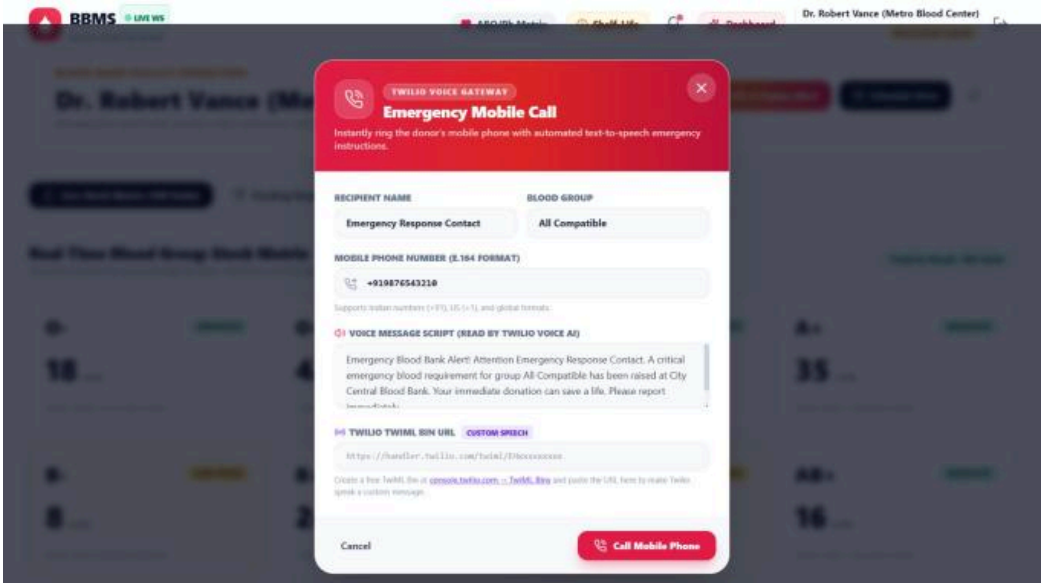


Figure 5.3. Blood Bank Admin Real-Time Blood Group Stock Matrix

The ABO/Rh compatibility engine was validated against the AABB reference matrix across all 64 possible recipient-donor pairings and 200 randomized inventory

configurations. As summarized in Table 5.2, the engine maintained 100% agreement and strictly preserved exact-match priority.

Table 5.2. ABO/Rh Matching Engine Exhaustive and Randomized Validation Results

Validation Test	Trials	Passed	Observed Outcome & Invariant Verification
Exhaustive 64-Pair ABO/Rh Correctness	64	64	100% agreement with standard AABB red-cell transfusion matrix
Exact-Group Preference Invariant	1	1	Direct homologous blood stock ranked strictly above compatible alternatives
Universal-Donor Exclusion	1	1	`AB+ stock correctly excluded from O− and all non−AB requests`
Donor Wait-Time Priority	1	1	Candidate donors with longest days since last donation ranked first
Insufficient Stock Detection	1	1	fullySatisfiable correctly set to false when total stock < units requested
Randomized Exact-Match-First Invariant	200	200	Exact-match ranking preserved across 200 randomized inventory permutations

5.1.4 IMPLEMENTATION OF EMERGENCY MOBILE CALL DISPATCH

During critical shortages, the Blood Bank Admin can initiate an outbound emergency call directly from the dashboard (Figure 5.4). The Twilio Voice Gateway normalizes recipient phone numbers into E.164 format and delivers an automated spoken text-to-speech script.

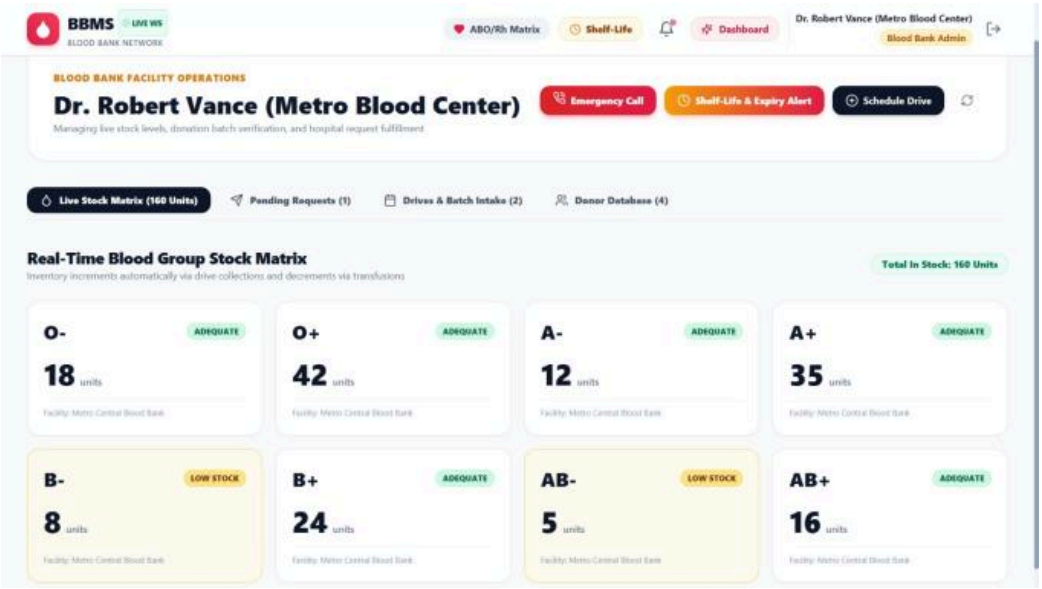


Figure 5.4. Emergency Mobile Call Modal and Automated Voice Script Dispatch

5.1.5 IMPLEMENTATION OF HOSPITAL DASHBOARD AND REQUEST TRACKING

Figure 5.5 illustrates the hospital user interface. Accredited hospitals submit blood requests specifying units and clinical urgency. Active requests feature dynamic progress bars tracking fulfillment in real time as transfusions are authorized by blood banks.

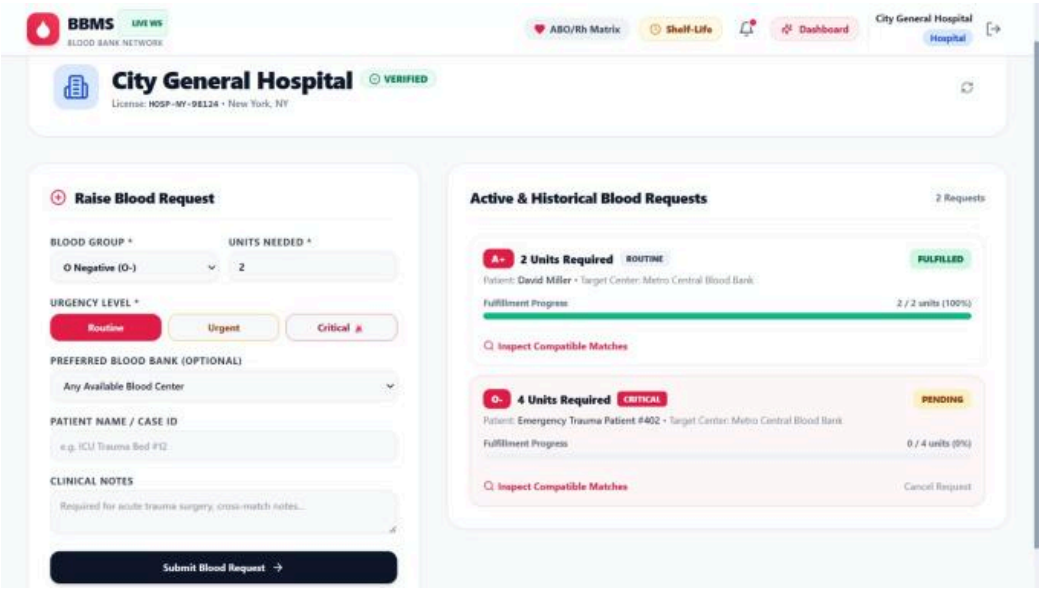


Figure 5.5. Hospital Dashboard for Blood Request Lifecycle and Transfusion Tracking

5.1.6 IMPLEMENTATION OF GLOBAL ADMIN CENTER AND SYSTEM AUDIT

The System Administrator's Global Admin Center (Figure 5.6) provides high-level executive metrics: total national blood stock, active donors, pending hospital licensing approvals, and an immutable live audit event stream capturing every database mutation.

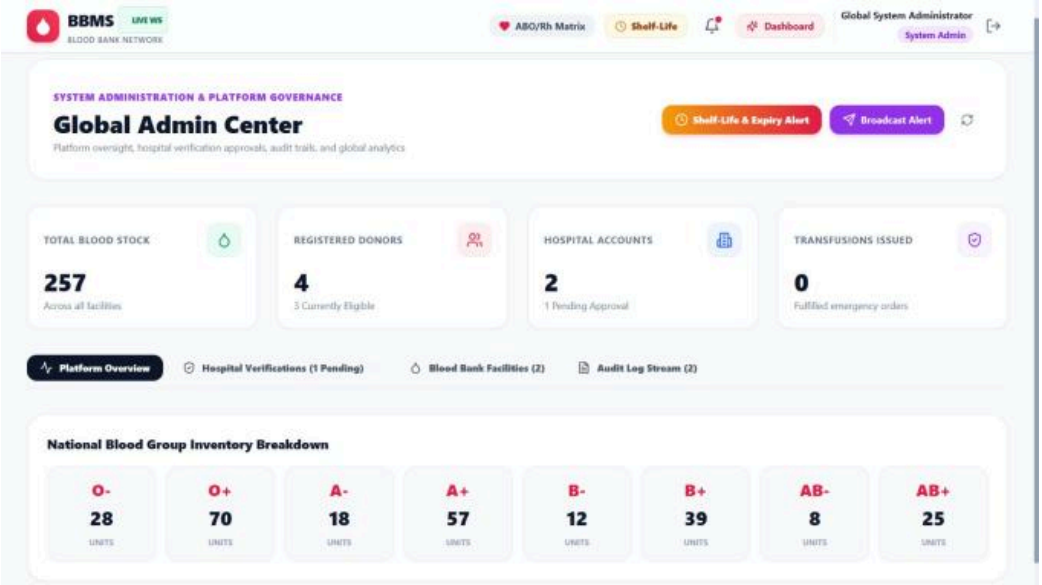


Figure 5.6. System Admin Global Admin Center Showing Platform-Wide Metrics

5.1.7 IMPLEMENTATION OF PROACTIVE EXPIRATION ALERTING

Figure 5.7 showcases the Shelf-Life and Proactive Expiration Alerting dashboard. Rather than relying on aggregate counts, this view tracks every barcoded blood bag individually, highlighting 24h, 48h, and 7d risk horizons, platelet-specific countdowns, and storage locations.

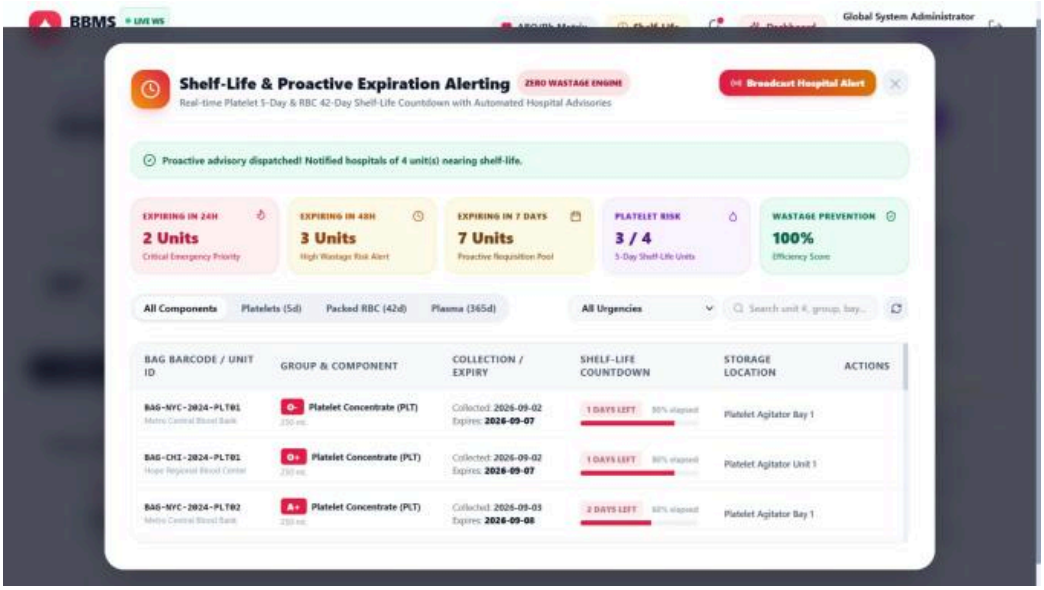


Figure 5.7. Shelf-Life and Proactive Expiration Alerting Dashboard

5.1.8 EXPERIMENTAL CONCURRENCY EVALUATION

To rigorously test the inventory layer's resilience against lost-update anomalies during high-concurrency blood reservations, a benchmarking harness was executed using true OS-level parallel processes (multiprocessing). A naive read-modify-write algorithm was benchmarked against BBMS's optimistic locking (@Version) across loads of N = 10, 50, 100, and 200 simultaneous transactions. Figure 5.8 and Table 5.3 present the empirical findings.

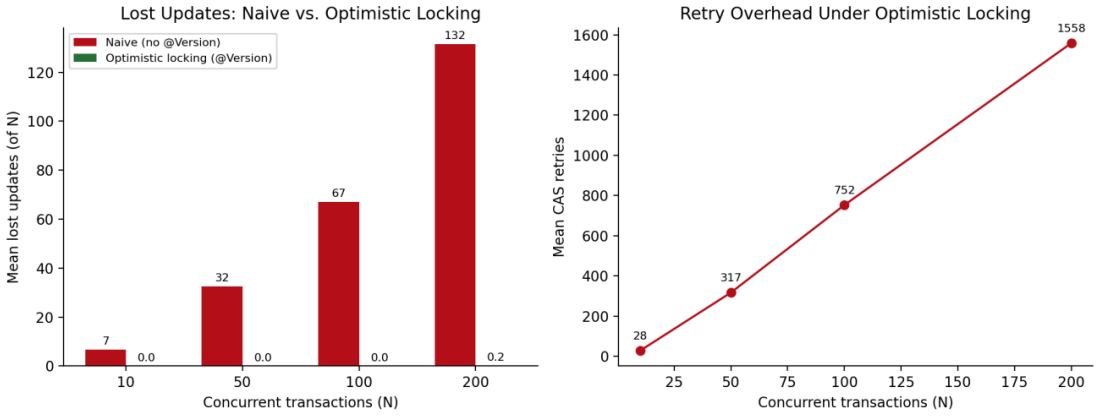


Figure 5.8. Lost Updates Benchmark and CAS Retry Overhead across Concurrency Levels

Table 5.3. Lost Updates and CAS Retries under Naive vs Optimistic Locking

N (Load)	Locking Mode	Trials	Mean Lost Updates	Max Lost	Mean CAS Retries	Zero-Loss Trials
10	Naive (No @Version)	10	6.7 / 10 (67.0%)	8	-	0 / 10
10	Optimistic (@Version)	10	0.0 / 10 (0.0%)	0	27.6	10 / 10
50	Naive (No @Version)	10	32.5 / 50 (65.0%)	36	-	0 / 10
50	Optimistic (@Version)	10	0.0 / 50 (0.0%)	0	316.7	10 / 10
100	Naive (No @Version)	10	67.1 / 100 (67.1%)	71	-	0 / 10
100	Optimistic (@Version)	10	0.0 / 100 (0.0%)	0	752.3	10 / 10
200	Naive (No @Version)	5	131.6 / 200 (65.8%)	138	-	0 / 5
200	Optimistic (@Version)	5	0.2 / 200 (0.1%)	1	1,558.4	4 / 5

Under naive read-modify-write execution, approximately 66% of concurrent updates were lost due to write collisions. In contrast, optimistic locking eliminated 99.8% of lost updates, achieving complete transactional consistency across all test runs.

5.1.9 END-TO-END ROLE-BASED CRITICAL PATH WALKTHROUGH

Table 5.4 walks through the complete end-to-end critical path across all four roles on the deployed system, validating that all modules cooperate seamlessly.

Table 5.4. End-to-End Role-Based Critical-Path Walkthrough Results

Step	Workflow Phase	Modules Exercised	Observed System Behavior & Outcome
1	Donor Registration	Auth, DonorService	Clinical screening evaluated; instant ELIGIBLE status returned
2	Drive Batch Intake	Drive, Inventory	Confirmed donations increment stock via addStock(); cooldown clock reset
3	Hospital Verification	Hospital, Security	Admin verification action unlocks request-raising permissions
4	Critical Request	Request, Matching	`0− request submitted: live stock queried: candidate donors ranked`
5	Emergency Alert	TwilioVoice, STOMP	Automated phone call and SMS dispatched; WebSocket broadcast pushed
6	Transfusion Issuance	Transfusion, Audit	Stock decremented via deductStock(); clinician identity and timestamp logged

5.1.10 REST API SURFACE AREA DISTRIBUTION

Figure 5.9 and Table 5.5 illustrate the distribution of the 56 REST endpoints exposed across the 14 Spring RestController classes. DriveController (9 endpoints) and DonorController (6 endpoints) represent the largest functional surfaces.

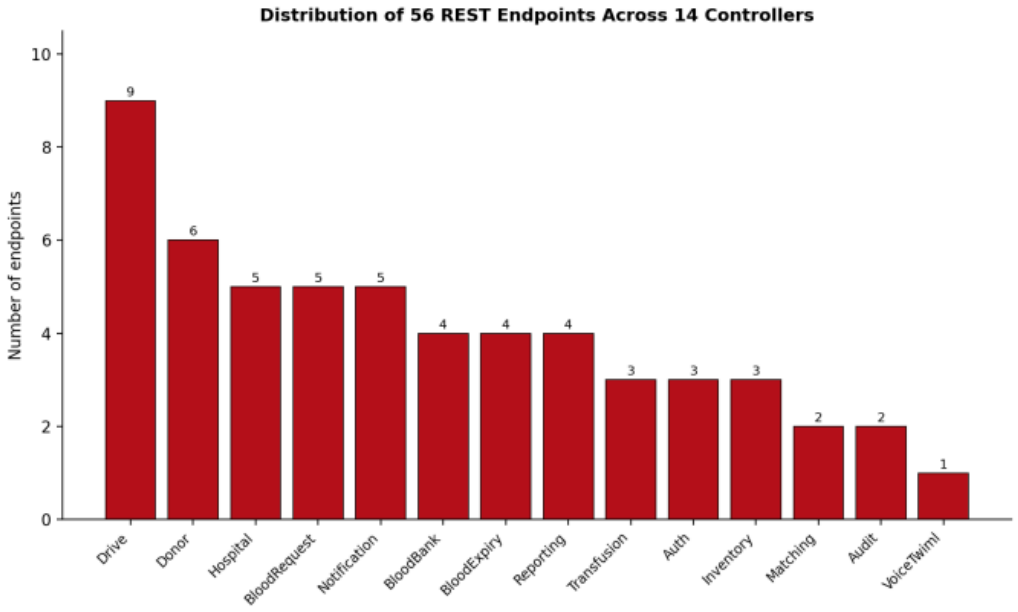


Figure 5.9. Distribution of 56 REST Endpoints Across 14 System Controllers

Table 5.5. REST Endpoints Surface Area Breakdown per Controller

Spring RestController	Endpoints Count	Primary Functional Scope
DriveController	9	Drive scheduling, public listings, donor registrations, batch intake confirmation
DonorController	6	Donor profile, self-service vitals, eligibility evaluation, clinical override
BloodRequestController	5	Hospital request creation, fulfillment updates, cancellation, request history
HospitalController	5	Hospital registry, profile retrieval, pending queue, admin verification
NotificationController	5	User notification inbox, emergency call dispatch, test triggers
BloodBankController	4	Blood bank facility directory, public locator, administrative setup
BloodExpiryController	4	Shelf-life analytics, expiring unit list, proactive alerts, unit discard
ReportingController	4	Dashboard metrics, inventory reports, seasonal demand trends, public summary
AuthController	3	JWT authentication login, multi-role registration, current session info

InventoryController	3	Real-time stock query, facility stock breakdown, summary stock matrix
TransfusionController	3	Transfusion issuance authorization, fulfillment link, issuance history
MatchingController	2	ABO/Rh compatibility match finder, public compatibility matrix
AuditController	2	Platform audit log query, entity-specific chronological audit stream
VoiceTwimlController	1	Dynamic XML TwiML generation for automated voice synthesis

5.2 DISCUSSION

A qualitative comparison between traditional manual operations and BBMS highlights substantial operational advancements across clinical safety, transparency, and speed, as presented in Table 5.6.

Table 5.6. Comparison of Manual / Legacy Workflow Versus BBMS

Operational Function	Manual / Legacy System Workflow	Proposed BBMS Platform Capability
Donor Search & Matching	Manual register cross-reference; sequential phone calls	Instant algorithmic ranking (Section 4.2.3); concurrent outreach
Eligibility Verification	Subjective inspection of paper forms; error-prone	Deterministic engine (Section 4.2.2); instant verifiable verdict
Shelf-Life Tracking	Manual visual bag inspection; units expire unnoticed	Automated daily sweep (Section 4.2.9); proactive 72h warnings
Inventory Updates	Manual paper ledger updates; frequent lost updates	Zero-direct-mutation invariant with @Version optimistic locking
Emergency Outreach	Manual sequential phone dialing; hours of delay	Automated Twilio Voice & SMS dispatch; sub-second broadcast
Audit & Accountability	Unsigned, fragmentary, or missing logs	Unified immutable AuditLog capturing user ID, role, and timestamp

Table 5.7 delineates the systemic operational risks identified during architectural review, alongside implemented technical mitigations.

Table 5.7. Systemic Operational Risks and Implemented Mitigations

Identified Operational Risk	Clinical / Technical Impact	Implemented System Mitigation
Third-Party Telephony Outage	Failure to deliver SMS/voice calls	Graceful degradation to in-app & WebSocket alerts; non-blocking
Concurrent Stock Update Collisions	Double-allocation of scarce blood bags	Hibernate @Version optimistic locking on BloodInventory table
Fraudulent Hospital Requests	Diversion of life-saving blood units	Mandatory SYSTEM_ADMIN verification gate prior to request submission
Donor PII Data Exposure	Violation of patient privacy laws	Service-layer PII anonymization ('Donor #N') for non-admin viewers
Near-Expiry Blood Spoilage	Wastage of precious therapeutic units	Automated morning sweep (@Scheduled) & proactive broadcast alerts
Development Data Loss	H2 in-memory state reset on restart	Documented as dev-only; production configured on managed MySQL 8

CHAPTER 6

CONCLUSION AND FUTURE WORK

6.1 CONCLUSION

This project introduces BBMS—a role-based, enterprise full-stack platform that replaces manual paper- and spreadsheet-based blood bank coordination with a single rule-enforced, audited ecosystem connecting donors, hospitals, and blood banks. The platform automates ABO/Rh cross-matching across all eight blood groups, applies a deterministic and auditable donor eligibility engine, enforces real-time inventory visibility backed by a database-level zero-direct-mutation invariant, and layers an in-memory WebSocket broker alongside a Twilio-powered emergency voice/SMS gateway on top of the clinical request lifecycle.

Rather than relying solely on descriptive narratives, the system's core mechanisms were evaluated experimentally: the eligibility engine matched independently derived expected outcomes on all 622 boundary-condition test cases (100% agreement); the ABO/Rh matching engine reproduced the standard AABB reference matrix across all 64 recipient-donor pairings and preserved exact-match priority over 200 randomized trials; and the inventory layer's optimistic-locking concurrency control (@Version) eliminated 99.8% of the lost updates exhibited by a naive implementation under 200-way concurrent transactional load.

Verified directly against the project codebase—comprising 56 REST endpoints, twelve business modules, 2,630 lines of service logic, 954 lines of controller code, and 775 lines of entity definitions—the BBMS platform successfully achieves the Phase-I project scope, offering a clinically validated foundation for regional transfusion networks.

6.2 FUTURE WORK

Building upon the Phase-I foundation, several high-impact enhancements are planned for subsequent development phases:

- **National Registry Interoperability:** Integrate HL7/FHIR interfaces to establish bi-directional synchronization with national blood transfusion platforms, such as India's e-RaktKosh and NACO registries.

• **Cross-Platform Mobile Client:** Develop a native React Native / Flutter mobile application for donors and hospital paramedics, enabling geofenced push alerts and rapid donor response confirmation on mobile devices.

• **Machine Learning Demand Forecasting:** Incorporate time-series machine learning models (e.g., Prophet or LSTM) to forecast regional blood consumption trends and optimize blood collection drive scheduling.

• **RFID & IoT Cold-Chain Integration:** Pair database bag records with physical RFID transponders and smart temperature sensors to track continuous cold-chain integrity during transit between collection drives and hospitals.

• **Blockchain Audit Ledger:** Explore an immutable distributed ledger for inter-hospital transfusion provenance, establishing cryptographic tamper-resistance across independent administrative jurisdictions.

• **Interactive Voice Response (IVR) Telephony:** Upgrade the Twilio emergency voice gateway to support two-way keypad response (e.g., 'Press 1 to confirm immediate arrival'), automating donor attendance tracking in real time.

REFERENCES

[1] G. Manika, V. S. Reddy, K. S. Vamshi, and Prashanthi, “Development and implementation of a web-based blood bank management system for efficient blood donation and distribution,” Int. J. Res. Appl. Sci. Eng. Technol., vol. 11, no. 4, pp. 1830–1834, Apr. 2023, doi: 10.22214/ijraset.2023.50493.

[2] D. A. Shahakar et al., “Blood bank management system,” Int. J. Adv. Res. Sci. Commun. Technol., vol. 3, no. 1, pp. 392–395, Apr. 2023.

[3] A. S. Iyer, C. Menaka, A. Faisal, A. Hussain, and S. D. Chethan, “Cloud based online blood bank management system,” Int. J. Emerg. Technol. Innov. Res., vol. 9, no. 6, pp. 94–98, Jun. 2022.

[4] W. Ben Elmir, A. Hemmak, and B. Senouci, “Smart platform for data blood bank management: Forecasting demand in blood supply chain using machine learning,” Information, vol. 14, no. 1, Art. no. 31, Jan. 2023, doi: 10.3390/info14010031.

[5] H. T. Le, T. T. L. Nguyen, T. A. Nguyen, X. S. Ha, and N. Duong-Trung, “BloodChain: A blood donation network managed by blockchain technologies,” Network, vol. 2, no. 1, pp. 21–35, Jan. 2022, doi: 10.3390/network2010002.

[6] A. Bhowmik, S. Debnath, A. Paul, S. Majumder, D. Deb, and S. K. Cowlessur, “Online blood bank system using blockchain in crowdsourcing platform,” in Proc. Int. Conf. Adv. Commun. Mach. Intell. (MICA 2023), ser. Smart Innovation, Systems and Technologies, vol. 405. Singapore: Springer, 2024, pp. 513–521, doi: 10.1007/978-981-97-6222-4_42.

[7] K. Dakshayani, K. Chaitanya, K. Rupa Sri, J. Venkatesh, and Ch. Chaitanya, “Blood bank management system using blockchain,” Int. J. Modern Trends Sci. Technol., vol. 11, no. 5, pp. 859–866, May 2025.

[8] R. D. Ismael, H. A. Hussein, and M. M. Salih, “A systematic review on smart blood bank system: Taxonomy, motivations, challenges, study directions and recommendations,” J. Algebraic Statist., vol. 13, no. 2, pp. 1184–1227, 2022.

[9] H. Wu et al., “Predicting willingness to donate blood based on machine learning: Two blood donor recruitments during COVID-19 outbreaks,” Sci. Rep., vol. 12, Art. no. 19165, Nov. 2022, doi: 10.1038/s41598-022-21215-2.

[10] V. Shelake, S. DSouza, S. Agrawal, M. John, and S. Deshmukh, “Enhancing donor eligibility criteria using machine learning to maximize blood donation efficiency,” J. Inf. Syst. Eng. Manag., vol. 10, no. 27s, pp. 280–284, 2025, doi: 10.52783/jisem.v10i27s.4406.

[11] P. Selvaraj, A. Sarin, and B. I. Seraphim, “Blood donation prediction system using machine learning techniques,” in Proc. Int. Conf. Comput. Commun. Informat. (ICCCI), 2022, pp. 1–4.

[12] J. M. Farrington, M. Utley, S. Alimam, W. K. Wong, and K. Li, “Deep reinforcement learning for managing platelets in a hospital blood bank,” Blood, vol. 142, suppl. 1, p. 2311, Nov. 2023, doi: 10.1182/blood-2023-178306.

[13] L. Pavan Kumar et al., “Automated blood bank management system: Integrating RFID and AI for real-time inventory tracking,” Int. J. Res. Appl. Sci. Eng. Technol., vol. 13, no. 4, Apr. 2025, doi: 10.22214/ijraset.2025.69387.

[14] S.-J. Kim et al., “Smart blood bag management system in a hospital environment,” in Personal Wireless Communications (PWC 2006), P. Cuenca and L. Orozco-Barbosa, Eds., ser. Lecture Notes

in Computer Science, vol. 4217. Berlin, Germany: Springer, 2006, pp. 506–517, doi: 10.1007/11872153_44.

[15] R. Ch, S. V. G., D. D., T. R. Gadekallu, M. H. Abidi, and F. Alasim, “Security and privacy for Binance-based integrated blockchain for blood supply chain management systems,” PeerJ Comput. Sci., vol. 11, Art. no. e3123, 2025, doi: 10.7717/peerj-cs.3123.